

Mettre OnLinePrep en ligne

Le guide pour un enfant de 5 ans

une étape par page · gros caractères
Site web (Vercel) · Base de données (Supabase) · Plans (GitHub) · Android · Apple

Ce qu'on va faire ensemble

Imagine que ton application est une maison. Aujourd'hui, la maison est dessinée sur du papier. On va la construire pour de vrai, puis la mettre dans une rue où tout le monde peut venir la visiter.

La maison a trois parties :

• **Le jardin devant** : c'est le site web que les élèves voient. Il vit chez **Vercel**.

• **La cave** : c'est là qu'on range les questions, les comptes et les scores. Elle vit chez **Supabase**.

• **Le coffre de plans** : c'est là qu'on garde tous les plans de la maison. Il vit chez **GitHub**.

Claude Code est ton **ouvrier**. Tu lui donnes des consignes (les « prompts »). Il construit. Toi, tu es le **patron** : c'est toi qui as les clés.

> ☺ **Pour un enfant de 5 ans**
> Tu n'as pas besoin de comprendre comment ça marche à l'intérieur. Tu as seulement besoin de suivre les étapes dans l'ordre.

Ta liste de courses

Avant de construire, il faut rassembler les choses. Coche chaque case quand tu l'as.

☐ Un **ordinateur** (Windows ou Mac) avec internet.

☐ Une **adresse e-mail** que tu utilises souvent.

☐ Un **téléphone** pour tester (Android de préférence).

☐ Un **compte GitHub** (gratuit) → étape 4.

☐ Un **compte Supabase** (gratuit pour commencer) → étape 7.

☐ Un **compte Vercel** (gratuit pour tester) → étape 11.

☐ Un abonnement **Claude** (pour utiliser Claude Code).

Plus tard, seulement pour les boutiques :

☐ **Google Play** : environ 25 dollars, une seule fois.

☐ **Apple** : environ 99 dollars par an (et un Mac, ou un service qui construit pour toi).

> ⚠ **Attention**
> Les prix des comptes changent. Regarde toujours le prix affiché sur le site avant de payer. C'est toi qui paies, jamais Claude.

> ★ **Astuce**
> Prends un carnet. Tu vas noter des mots de passe. Le carnet reste chez toi.

A Les mots magiques

Quelques mots que tu vas rencontrer. Pas besoin de les apprendre par cœur : reviens ici quand tu en as besoin.

Terminal : une fenêtre où tu écris des ordres à l'ordinateur. Comme un talkie-walkie.

Commande : une phrase que tu écris dans le terminal, puis tu appuies sur Entrée.

Dépôt (repo) : un dossier dans le coffre GitHub où vivent tous les plans.

Commit : prendre une photo des plans à un moment précis. On peut toujours revenir à une photo.

Prompt : la consigne que tu donnes à Claude Code.

Clé secrète : un mot de passe pour les machines. Elle ne se montre à personne.

Variable d'environnement : un petit tiroir où l'on range une clé secrète, loin du regard.

Déployer : mettre la maison dans la rue, visible par tous.

Migration : une consigne qui range la cave (la base de données) dans le bon ordre.

> ☺ **Pour un enfant de 5 ans**
>
> Si un mot t'embrouille, demande à Claude : « Explique-moi ce mot comme si j'avais 5 ans. »

1 Étape 1 — Installer Node (le moteur)

Node est le moteur qui fait tourner ton application sur ton ordinateur. On l'installe une seule fois.

1. Ouvre ton navigateur. Va sur **nodejs.org**.

2. Clique sur le gros bouton marqué **LTS**. LTS veut dire « version stable ».

3. Ouvre le fichier téléchargé. Clique sur **Suivant**, **Suivant**, **Installer**.

4. Quand c'est fini, ouvre le terminal : sur Windows, cherche **PowerShell** ; sur Mac, cherche **Terminal**.

5. Écris la commande ci-dessous, puis appuie sur Entrée.

```
node --version
```

✔ C'est réussi si…

Tu vois un numéro qui commence par v22 ou plus. Si tu vois « commande introuvable », ferme le terminal, rouvre-le, et réessaie.

2 Étape 2 — Installer Git et pnpm

Git prend des photos de tes plans. pnpm est le livreur qui apporte les petits morceaux dont l'application a besoin.

1. Va sur **git-scm.com** et télécharge Git pour ton ordinateur.

2. Installe-le : à chaque question, laisse les réponses proposées et clique sur **Suivant**.

3. Dans le terminal, écris ces lignes (avec ton nom et ton e-mail), une par une :

```
git --version
git config --global user.name "Ton Nom"
git config --global user.email "ton@email.com"
corepack enable
pnpm --version
```

> **✔ C'est réussi si…**
>
> « git --version » répond avec un numéro de version. Un numéro s'affiche. Si on te dit « permission refusée » sur Mac, écris « sudo corepack enable » puis ton mot de passe d'ordinateur (il ne s'affiche pas quand tu l'écris, c'est normal).

3 Étape 3 — Installer Claude Code (ton ouvrier)

Claude Code est l'ouvrier qui va construire. Il lit tes consignes et écrit le code.

1. Dans le terminal, écris la première commande. Elle installe l'ouvrier.

2. Ensuite écris « claude » : une fenêtre s'ouvre et te demande de te connecter.

3. Suis les instructions à l'écran : tu te connectes avec ton compte Claude dans le navigateur.

```
npm install -g @anthropic-ai/claude-code
claude --version
```

✔ C'est réussi si…

Un numéro de version s'affiche, et quand tu écris « claude » dans un dossier, l'ouvrier te dit bonjour.

★ Astuce

Si la méthode d'installation change, cherche « Claude Code installation » sur le site officiel d'Anthropic : la version à jour y est toujours.

4 Étape 4 — Ouvrir le coffre de plans (GitHub)

GitHub est le coffre où tes plans sont gardés en sécurité, même si ton ordinateur tombe en panne.

1. Va sur **github.com**. Clique sur **Sign up** (S'inscrire).

2. Choisis un nom d'utilisateur, un e-mail, un mot de passe solide. Note-les dans ton carnet.

3. Ouvre l'e-mail de GitHub et clique sur le lien pour confirmer.

4. Active la **double sécurité** (2FA) dans Réglages → Mot de passe et authentification. GitHub te demandera un code de ton téléphone à chaque connexion : c'est un cadenas en plus.

> ✔ **C'est réussi si…**
>
> Tu vois ton tableau de bord GitHub, et le dépôt de ton projet est visible.

> ⚠ **Attention**
>
> Ne dis jamais ton mot de passe GitHub à personne, pas même à un « conseiller » qui t'écrit sur WhatsApp.

5 Étape 5 — Télécharger les plans sur ton ordinateur

Maintenant on copie les plans du coffre vers ton ordinateur. Ça s'appelle « cloner ».

1. Sur GitHub, ouvre ton dépôt. Clique sur le bouton vert **Code**, puis copie l'adresse.

2. Dans le terminal, écris « git clone » puis colle l'adresse. Voici la forme :

3. Entre dans le dossier, puis lance l'ouvrier :

```
git clone https://github.com/TON-NOM/TON-DEPOT.git
cd TON-DEPOT
claude
```

✔ C'est réussi si…

Le dossier contient des sous-dossiers comme design-system, docs et replica. Tu les vois avec « ls » (Mac) ou « dir » (Windows).

6 Étape 6 — Donner les consignes à l'ouvrier

C'est le moment magique. On donne les prompts à Claude Code, un par un.

1. Ouvre le fichier **docs/PROMPTS_CLAUDE_CODE.md**.

2. Copie le **bloc P0** et colle-le dans un fichier nommé **CLAUDE.md** à la racine du dossier (c'est le règlement de la maison).

3. Copie le **prompt P1**. Colle-le dans Claude Code. Appuie sur Entrée. Attends qu'il finisse.

4. Quand il dit que tout est vert (les tests passent), colle **P2**. Puis P3, P4… dans l'ordre.

5. Après chaque prompt, colle la « boucle d'amélioration » en bas du fichier : l'ouvrier vérifie son propre travail.

> ✔ **C'est réussi si…**
> Après P1, la commande « pnpm build » finit sans message rouge.

> ⚠ **Attention**
> Un prompt à la fois. Ne saute pas d'étape. Si l'ouvrier te pose une question, réponds : c'est toi le patron.

7 Étape 7 — Ouvrir la cave (Supabase) et créer ta cave de test

Supabase est la cave : elle garde les comptes, les questions et les réponses des élèves. On commence par une cave pour jouer.

1. Va sur **supabase.com**. Clique sur **Start your project**.

2. Connecte-toi avec ton compte GitHub (le plus simple).

3. Crée une **organisation** (un nom, par exemple OnLinePrep).

4. Accepte les conditions. Tu arrives sur le tableau de bord.

1. Clique sur **New project**. Nom : **onlineprep-dev**.

2. Choisis un **mot de passe de base de données** très long. Clique sur le bouton qui en génère un. **Copie-le dans ton carnet.**

3. Choisis la **région** la plus proche de tes élèves parmi celles proposées.

4. Clique sur **Create new project**. Attends 2 minutes.

> **✔ C'est réussi si…**
> Tu vois un tableau de bord Supabase avec un bouton **New project**. Le projet affiche un tableau de bord prêt à l'emploi.

> **⚠ Attention**
> Le mot de passe de la base ne se récupère pas facilement. Si tu le perds, il faudra en changer.

8 Étape 8 — Les clés de la cave

Une clé permet à l'application d'entrer dans la cave. Il y en a deux sortes, et il faut bien les distinguer.

• **La clé publique** (« anon » ou « publishable ») : elle peut être vue dans le site web. Elle n'ouvre que ce que les règles de sécurité autorisent.

• **La clé secrète** (« service_role » ou « secret ») : elle ouvre TOUT. Elle ne doit jamais apparaître dans le site, dans GitHub, dans un message, ni dans une capture d'écran.

Où les trouver : Supabase → ton projet → **Project Settings** → **API**.

Où les ranger : dans un fichier nommé **.env.local** à la racine de l'application. Ce fichier n'est jamais envoyé à GitHub. Il ressemble à ça :

```
NEXT_PUBLIC_SUPABASE_URL=adresse-du-projet
NEXT_PUBLIC_SUPABASE_ANON_KEY=cle-publique
SUPABASE_SERVICE_ROLE_KEY=cle-secrete
```

✔ C'est réussi si…

Le fichier .env.local existe et git l'ignore. Demande à Claude : « Vérifie que .env.local est bien ignoré par git ».

⚠ Attention

Si tu colles une clé secrète dans un chat, change-la tout de suite dans Supabase.

9 Étape 9 — Ranger la cave (les migrations)

La cave est vide. Les migrations sont des consignes qui construisent les étagères : tables, règles de sécurité, tout.

1. Demande à Claude : « Installe la CLI Supabase pour mon système ».

2. Connecte-toi (une page s'ouvre dans le navigateur).

3. Relie ton dossier à ta cave de test. Le « ref » est la suite de lettres dans l'adresse de ton projet.

4. Envoie les étagères dans la cave.

```
supabase login
supabase link --project-ref TON-REF
supabase db push
```

> **✔ C'est réussi si…**
> Dans Supabase → **Table Editor**, tu vois des tables comme questions, attempts, entitlements. Chaque table a des règles de sécurité (RLS).

10 Étape 10 — Allumer l'application chez toi

Avant de la montrer au monde, on l'allume chez toi pour vérifier que tout marche.

1. Installe les petits morceaux nécessaires (une seule fois).

2. Allume l'application.

3. Ouvre ton navigateur et va à l'adresse indiquée (en général **localhost:3000**).

```
pnpm install
pnpm dev
```

✔ C'est réussi si…

Tu vois la page d'accueil OnLinePrep, bleu marine et or, avec le logo. Va aussi sur /design pour voir tous les boutons.

★ Astuce

Pour éteindre l'application : appuie sur Ctrl et C en même temps dans le terminal.

11 Étape 11 — Ouvrir le jardin (Vercel) et importer les plans

Vercel est le jardin devant la maison : c'est là que ton site devient visible sur internet.

1. Va sur **vercel.com**. Clique sur **Sign Up**.

2. Choisis **Continue with GitHub**. Accepte.

3. Vercel te demande un nom d'équipe. Choisis le plan **Hobby** pour essayer.

1. Dans Vercel, clique sur **Add New → Project**.

2. Dans la liste, trouve ton dépôt et clique sur **Import**.

3. **Root Directory** : choisis **apps/web** (c'est le dossier du site).

4. Ne clique pas encore sur Deploy : il manque les tiroirs à clés. Étape suivante.

> **✔ C'est réussi si…**
>
> Tu vois le tableau de bord Vercel avec un bouton **Add New… → Project**. Vercel a reconnu **Next.js** tout seul (« Framework Preset : Next.js »).

> ⚠ **Attention**
>
> Le plan Hobby est fait pour des essais personnels, pas pour vendre. Quand tu commences à encaisser de l'argent, passe au plan Pro (regarde le prix sur la page « Pricing »).

12 Étape 12 — Mettre les clés dans les tiroirs

Vercel a des tiroirs secrets : les « Environment Variables ». C'est là que vont les clés, jamais dans GitHub.

1. Sur la page d'import, ouvre **Environment Variables**.

2. Ajoute une ligne par variable : le **nom** à gauche, la **valeur** à droite.

3. Les noms sont ceux du fichier **.env.example** du projet :

4. Pour la production, utilise la clé de ta cave de **production**, pas celle de test.

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
PAYMENT_PROVIDER
PAYMENT_WEBHOOK_SECRET
SMS_PROVIDER
```

✔ C'est réussi si…

Chaque nom est dans la liste de Vercel, et la clé secrète n'est affichée nulle part en clair.

⚠ Attention

Les variables qui commencent par NEXT_PUBLIC_ sont visibles par tout le monde. Ne mets JAMAIS une clé secrète dans une variable qui commence par NEXT_PUBLIC_.

13 Étape 13 — Mettre la maison dans la rue

Le grand moment : on déploie.

1. Clique sur **Deploy**.

2. Attends 1 à 3 minutes : Vercel construit le site.

3. Quand des confettis apparaissent, clique sur l'image de ton site : l'adresse ressemble à **onlineprep-xxxx.vercel.app**.

4. Désormais, chaque fois que l'ouvrier envoie un changement sur la branche **main**, Vercel met le site à jour tout seul.

> **✔ C'est réussi si…**
> Tu ouvres l'adresse sur ton téléphone, en 4G, et tu vois OnLinePrep.

> **★ Astuce**
> S'il y a un message rouge, clique sur « View build logs », copie les dernières lignes et colle-les dans Claude Code : « Corrige cette erreur de déploiement ».

14 Étape 14 — Tester avec ton téléphone

Un site, ça se teste avec ses doigts. Prends ton téléphone et fais le parcours d'un élève.

☐ Ouvre le site. Il s'affiche vite (moins de 4 secondes).

☐ Crée un compte avec ton numéro. Le code arrive.

☐ Fais le diagnostic de 15 questions.

☐ Mets le téléphone en **mode avion**. Réponds à 3 questions : l'application doit dire que tes réponses sont gardées sur le téléphone.

☐ Enlève le mode avion : les réponses partent toutes seules.

☐ Agrandis le texte du téléphone : rien ne doit disparaître.

☐ Passe en mode sombre : tout doit rester lisible.

> **✔ C'est réussi si…**
>
> Aucune réponse n'a été perdue. Si une réponse disparaît, c'est un bug grave : dis-le à Claude Code.

15 Étape 15 — Ajouter tes propres questions

Une application vide ne sert à rien. Les questions, c'est le trésor.

1. Va sur l'adresse **ton-site/admin**.

2. Connecte-toi avec ton compte administrateur (demande à Claude de t'en créer un).

3. Clique sur **Nouvelle question**. Écris l'énoncé, les réponses, la bonne réponse et l'explication.

4. Clique sur **Soumettre pour relecture**. Une AUTRE personne doit la relire, puis une troisième la valider. Une question ne se publie jamais toute seule.

> **✔ C'est réussi si…**
> Une question « Publiée » apparaît dans l'entraînement d'un compte élève.

> **★ Astuce**
> Pour beaucoup de questions d'un coup : prépare un fichier CSV et utilise l'import. L'outil montre les erreurs ligne par ligne et n'importe rien s'il y a un problème.

16 Étape 16 — Donner un vrai nom à ta maison

L'adresse vercel.app marche, mais une adresse comme onlineprep.cm est plus belle. C'est facultatif au début.

1. Achète un nom de domaine chez un vendeur de ton choix (c'est toi qui paies et qui gardes le compte).

2. Dans Vercel → ton projet → **Settings → Domains**, écris ton nom de domaine.

3. Vercel te montre 1 ou 2 lignes à recopier chez ton vendeur de domaine (des « enregistrements DNS »).

4. Recopie-les exactement. Attends quelques minutes à quelques heures.

> ✔ **C'est réussi si…**
> Ton nom de domaine ouvre le site avec un petit cadenas (HTTPS).

> ⚠ **Attention**
> Le compte du domaine doit être à ton nom ou à celui de ton entreprise. Si un prestataire l'achète à son nom, tu peux perdre ton adresse.

17 Étape 17 — Construire l'application Android

Le site marche dans le navigateur. L'application Android se télécharge dans Google Play. On la construit avec un outil nommé EAS.

1. Crée un compte gratuit sur **expo.dev**.

2. Dans le terminal, dans le dossier de l'application mobile, connecte-toi.

3. Pour un fichier à tester sur ton téléphone (APK) :

4. Pour le fichier destiné à Google Play (AAB) :

```
npx eas-cli login
npx eas-cli build --platform android --profile preview
npx eas-cli build --platform android --profile production
```

✔ C'est réussi si…

EAS te donne un lien. Tu l'ouvres sur ton téléphone Android et tu installes OnLinePrep.

★ Astuce

Teste d'abord sur le téléphone le plus faible que tu connais (2 Go de mémoire). S'il tient, les autres tiendront.

18 Étape 18 — Publier sur Google Play

Google Play est le grand magasin d'applications Android. Il vérifie ton application avant de l'accepter.

1. Va sur **play.google.com/console** et crée un compte développeur (paiement unique, regarde le prix affiché).

2. Crée l'application : nom, langue (Français), gratuite ou payante.

3. Remplis la fiche : description, captures d'écran, **l'icône 512 px** (dossier design-system/assets/icons), image de présentation (logo avec slogan).

4. Remplis le questionnaire **Sécurité des données** avec exactitude et ajoute l'adresse de ta politique de confidentialité.

5. Envoie le fichier AAB en **test fermé**. Pour un compte personnel récent, Google peut demander un nombre minimum de testeurs pendant environ 14 jours : lis ce que ta console demande.

6. Quand le test est accepté, demande la **production**.

> ✔ **C'est réussi si…**
> L'état de l'application devient « Disponible sur Google Play ».

> ⚠ **Attention**
> Si ton application vend des contenus numériques, lis la règle des paiements de Google AVANT de demander la publication. Un refus coûte du temps.

19 Étape 19 — Publier sur l'App Store (Apple)

Apple est plus strict et plus cher. On le fait en dernier, quand Android marche déjà.

1. Inscris-toi à l'**Apple Developer Program** (abonnement annuel, regarde le prix affiché).

2. Construis l'application iPhone avec EAS : « eas build --platform ios ».

3. Envoie-la dans **TestFlight** (« eas submit --platform ios ») pour tester sur de vrais iPhone.

4. Dans **App Store Connect**, remplis la fiche, les captures, la confidentialité, et le mode d'achat.

5. Soumets pour la **revue**. Apple répond en général en quelques jours.

> ✔ **C'est réussi si…**
> Un testeur installe l'application via TestFlight et fait le parcours complet.

> ⚠ **Attention**
> Apple refuse les applications qui ne sont qu'un site web emballé. C'est pourquoi on construit une vraie application avec Expo.

? Si ça ne marche pas

Tout le monde se trompe. Voici quoi faire, dans l'ordre.

1. **Respire.** Presque tout se répare.

2. Lis le message en rouge. Il dit souvent ce qui manque (une clé, un nom mal écrit).

3. Copie le message en entier. Colle-le dans Claude Code avec : « Voici l'erreur, explique-la comme à un enfant puis corrige. »

4. Si Vercel échoue : ouvre **Deployments → le dernier → Build Logs**.

5. Si le site s'ouvre mais ne trouve pas les données : les clés dans Vercel sont fausses ou manquantes.

6. Si tu as cassé quelque chose : « git log » montre les photos ; demande à Claude de revenir à la dernière photo qui marchait.

7. Si tu as montré une clé secrète à quelqu'un : change-la **tout de suite** dans Supabase.

> **★ Astuce**
>
> Ne supprime jamais le projet Supabase de production pour « recommencer » : tu perdrais les comptes et les réponses des élèves.

✔ Ta liste finale avant d'ouvrir au public

Avant de dire « c'est ouvert ! », on vérifie tout. Chaque case doit être cochée.

☐ Le site s'ouvre sur un téléphone en 4G.

☐ Une réponse donnée hors ligne arrive après le retour du réseau.

☐ Les clés secrètes ne sont dans aucun fichier du coffre GitHub.

☐ La double sécurité est active sur GitHub, Supabase, Vercel et Google.

☐ La politique de confidentialité et les conditions sont en ligne, en français.

☐ On peut supprimer son compte depuis l'application.

☐ Les questions publiées ont été relues par une autre personne.

☐ Le paiement a été testé avec un vrai petit paiement, puis remboursé.

☐ Tu connais le numéro à appeler si ça tombe en panne.

☐ Tu as une copie de sauvegarde des questions.

✔ C'est réussi si…

Toutes les cases sont cochées. Bravo : tu as construit et publié une vraie application. Tu es le patron de la maison.

☺ Pour un enfant de 5 ans

Et maintenant, on continue d'améliorer : écoute les élèves, corrige, recommence.